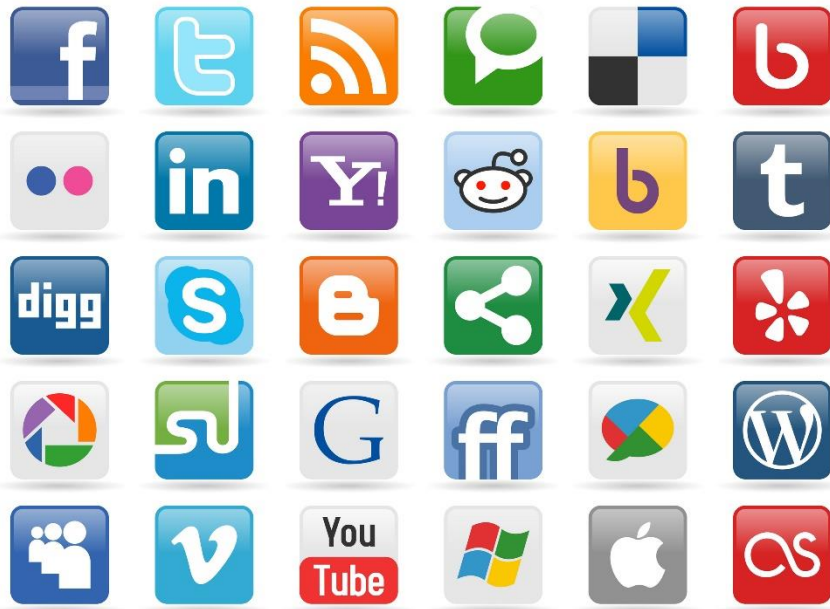




**Community
Analysis**

SOCIAL

MEDIA

MINING



Social Community



[real-world] community

A group of individuals with common *economic*, *social*, or *political* interests or characteristics, often living in *relative proximity*.

Why analyze communities?

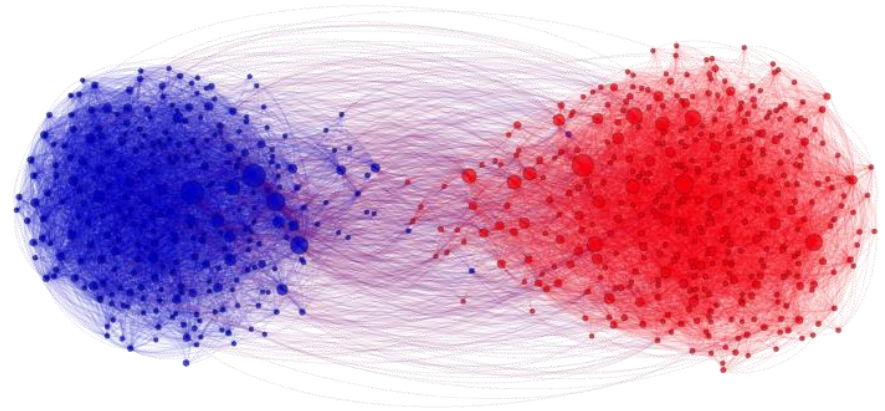


Analyzing communities helps better understand users

- Users form groups based on their interests

Groups provide a clear global view of user interactions

- E.g., find polarization



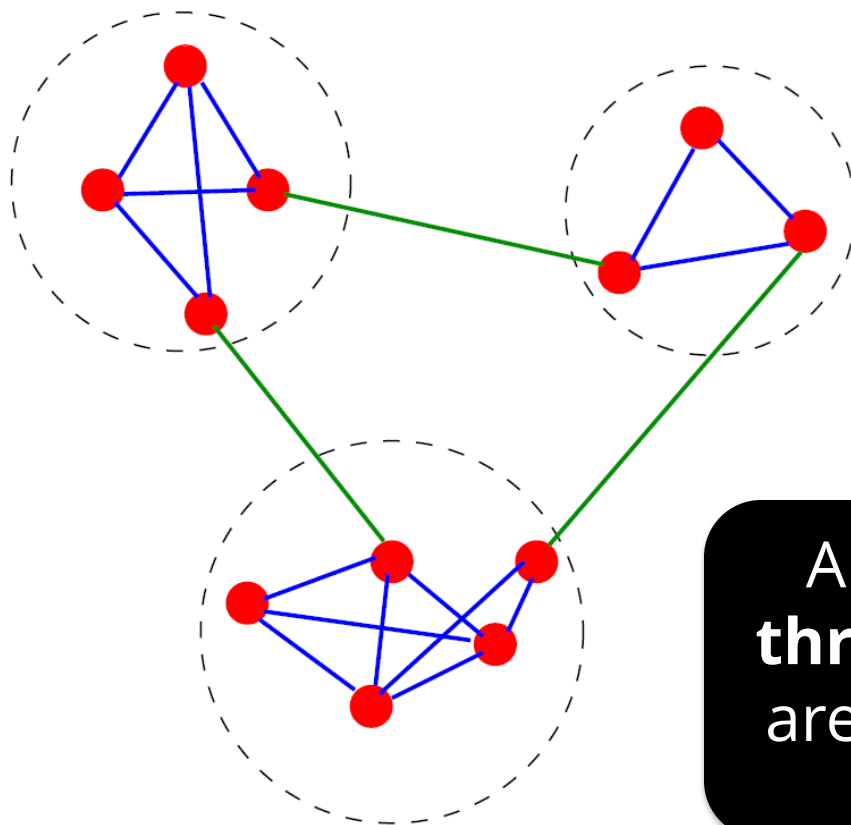
World Wide Web

- Communities may correspond to groups of pages dealing with the same or related topics

Social Media Communities

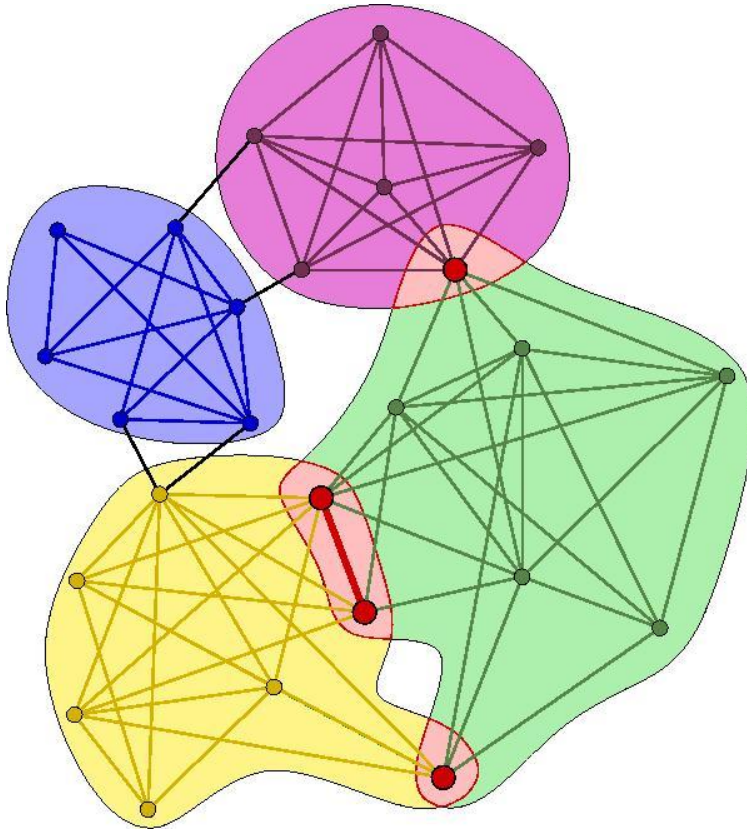
- **Formation:**
 - When like-minded users on social media form a link and start interacting with each other
- **More Formal Formation:**
 1. A set of at least two nodes sharing some interest, and
 2. Interactions with respect to that interest.
- Social Media Communities
 - **Explicit** (*emic*): formed by user subscriptions
 - **Implicit** (*etic*): implicitly formed by social interactions
 - **Example:** individuals calling Canada from the United States
 - Phone operator considers them one community for promotional offers
- Other community names: *group*, *cluster*, *cohesive subgroup*, or *module*

Finding Implicit Communities: An Example

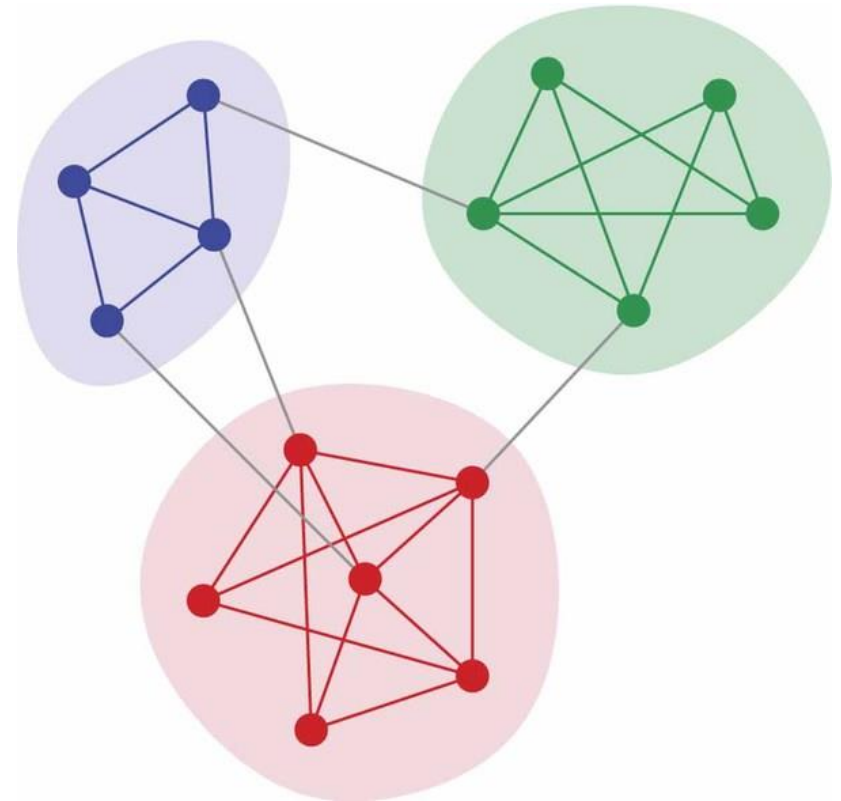


A simple graph in which **three** implicit communities are found, enclosed by the dashed circles

Overlapping vs. Disjoint Communities



Overlapping Communities



Disjoint Communities

What is Community Analysis?

- **Community detection**
 - Discovering implicit communities
- **Community evolution**
 - Studying temporal evolution of communities
- **Community evaluation**
 - Evaluating Detected Communities

Community Detection

What is community detection?

- The process of finding clusters of nodes (*"communities"*)
 - With **Strong** internal connections and
 - **Weak** connections between different communities
- Ideal decomposition of a large graph
 - Completely disjoint communities
 - There are no interactions between different communities.
- In practice,
 - find community partitions that are maximally decoupled.

Community Detection vs. Clustering

Clustering

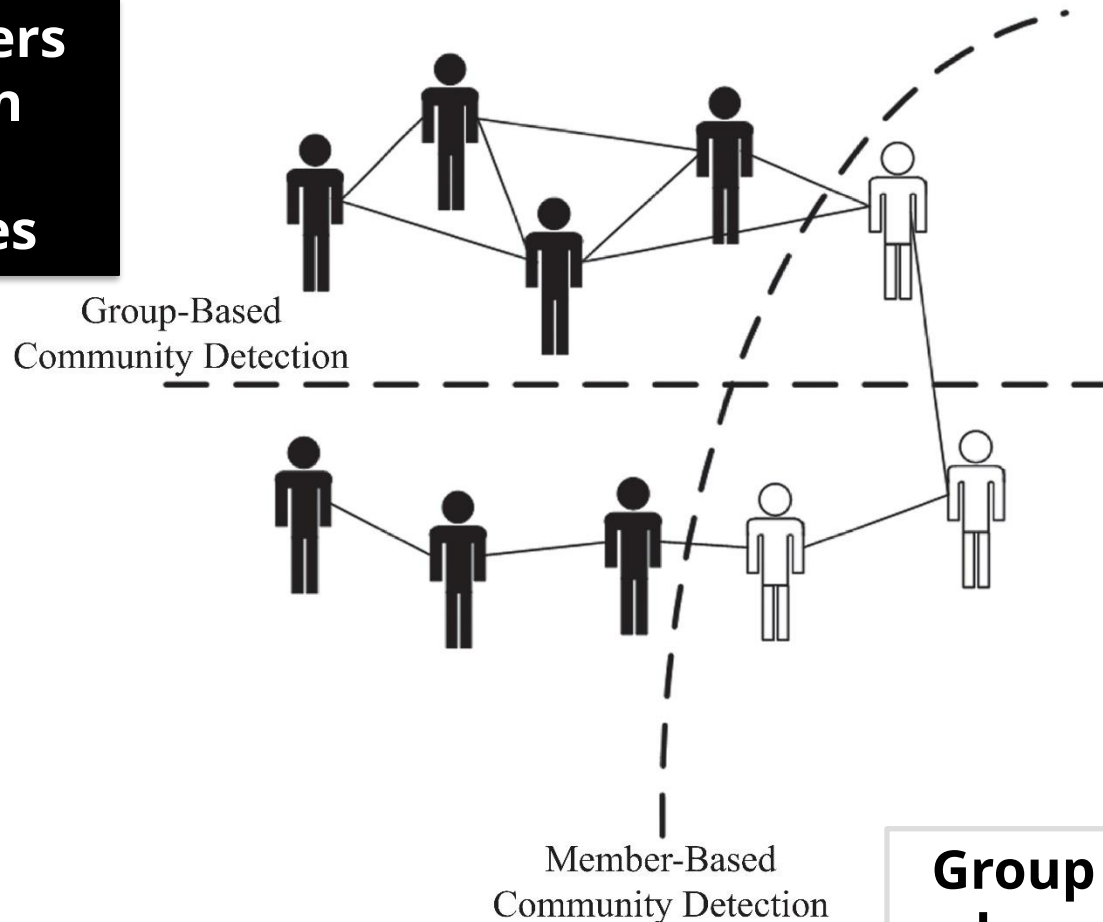
- Clustering works on the distance or similarity matrix, e.g., k -means.
- Using k -means with adjacency matrix rows, only consider the ego-centric network

Community detection

- Data is linked (a graph)
- Network data tends to be “discrete”, leading to algorithms using the graph property directly
 - k -clique, quasi-clique, or edge-betweenness

Community Detection Algorithms

**Group Users
based on
Group
attributes**



**Group Users
based on
Member
attributes**

Member-Based Community Detection

Member-Based Community Detection

- Look at node characteristics; and
- Identify nodes with similar characteristics and consider them a community

Node Characteristics

A. Degree

- Nodes with same (or similar) degrees are in one community
- Example: cliques

B. Reachability

- Nodes that are close (small shortest paths) are in one community
- Example: k -cliques, k -clubs, and k -clans

C. Similarity

- Similar nodes are in the same community
- Using the similarity of node neighborhoods: Jaccard, cosine

Group-Based Community Detection

Group-Based Community Detection

Group-based community detection: finding communities that have certain **group properties**

Group Properties:

- I. **Modular:** Modularity Maximization
- II. **Hierarchical:** Hierarchical clustering

Modular Communities

Consider a graph $G(V, E)$, where the degrees are known beforehand however edges are not

- Consider two vertices v_i and v_j with degrees d_i and d_j .

What is an expected **number of edges** between v_i and v_j ?

- For any edge going out of v_i randomly the probability of this edge getting connected to vertex v_j is

$$\frac{d_j}{\sum_i d_i} = \frac{d_j}{2m}$$

Modularity and Modularity Maximization

- Given a degree distribution, we know the expected number of edges between any pairs of vertices
- We assume that real-world networks should be far from random. Therefore, the more distant they are from this randomly generated network, the more structural they are.
- Modularity defines this distance and modularity maximization tries to maximize this distance

Normalized Modularity

Consider a partitioning of the data $P = (P_1, P_2, P_3, \dots, P_k)$

For partition P_x , this distance can be defined as

$$\sum_{i,j \in P_x} A_{ij} - \frac{d_i d_j}{2m}$$

This distance can be generalized for a partitioning P

$$\sum_{x=1}^k \sum_{i,j \in P_x} A_{ij} - \frac{d_i d_j}{2m}$$

The normalized version of this distance is defined as **Modularity**

$$Q = \frac{1}{2m} \sum_{x=1}^k \sum_{i,j \in P_x} A_{ij} - \frac{d_i d_j}{2m}$$

Modularity Expression

$$Q = \frac{1}{2m} \sum_{x=1}^k \sum_{i,j \in P_x} A_{ij} - \frac{d_i d_j}{2m}$$

$$\delta(x, y) = \begin{cases} 0, & \text{if } x \neq y \\ 1, & \text{if } x = y \end{cases}$$

Modularity can be restated as **Kronecker delta** function

$$Q = \frac{1}{2m} \sum_{i=1}^n \sum_{j=1}^n \left[A_{ij} - \frac{d_i d_j}{2m} \right] \delta(P_i, P_j)$$

partition of node i

Modularity Maximization

Modularity matrix

$$B = A - dd^T / 2m$$

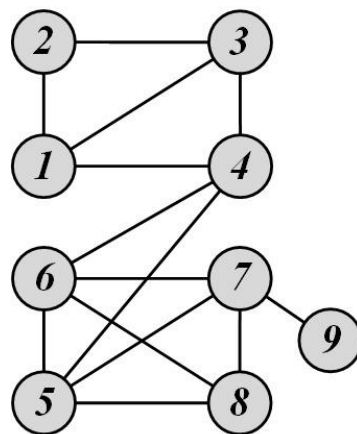
$d \in \mathbb{R}^{n \times 1}$ is the degree vector for all nodes

Reformulation of the modularity

$$Q = \frac{1}{2m} \text{Tr}(X^T B X)$$

- $X \in \mathbb{R}^{n \times k}$ is the indicator (partition membership) function:
 - $X_{ij} = 1$ iff. $v_i \in P_j$
- Maximizing modularity is NP-hard

Modularity Maximization: Example



**Two
Communities:**
 {1, 2, 3, 4}
 and
 {5, 6, 7, 8, 9}

$$B = A - dd^T / 2m$$

$$B_{ij} = A_{ij} - d_i d_j / 2m$$



$$B = \begin{bmatrix} -0.32 & 0.79 & 0.68 & 0.57 & -0.43 & -0.43 & -0.43 & -0.32 & -0.11 \\ 0.79 & -0.14 & 0.79 & -0.29 & -0.29 & -0.29 & -0.29 & -0.21 & -0.07 \\ 0.68 & 0.79 & -0.32 & 0.57 & -0.43 & -0.43 & -0.43 & -0.32 & -0.11 \\ 0.57 & -0.29 & 0.57 & -0.57 & 0.43 & 0.43 & -0.57 & -0.43 & -0.14 \\ -0.43 & -0.29 & -0.43 & 0.43 & -0.57 & 0.43 & 0.43 & 0.57 & -0.14 \\ -0.43 & -0.29 & -0.43 & 0.43 & 0.43 & -0.57 & 0.43 & 0.57 & -0.14 \\ -0.43 & -0.29 & -0.43 & -0.57 & 0.43 & 0.43 & -0.57 & 0.57 & 0.86 \\ -0.32 & -0.21 & -0.32 & -0.43 & 0.57 & 0.57 & 0.57 & -0.32 & -0.11 \\ -0.11 & -0.07 & -0.11 & -0.14 & -0.14 & -0.14 & 0.86 & -0.11 & -0.04 \end{bmatrix}$$

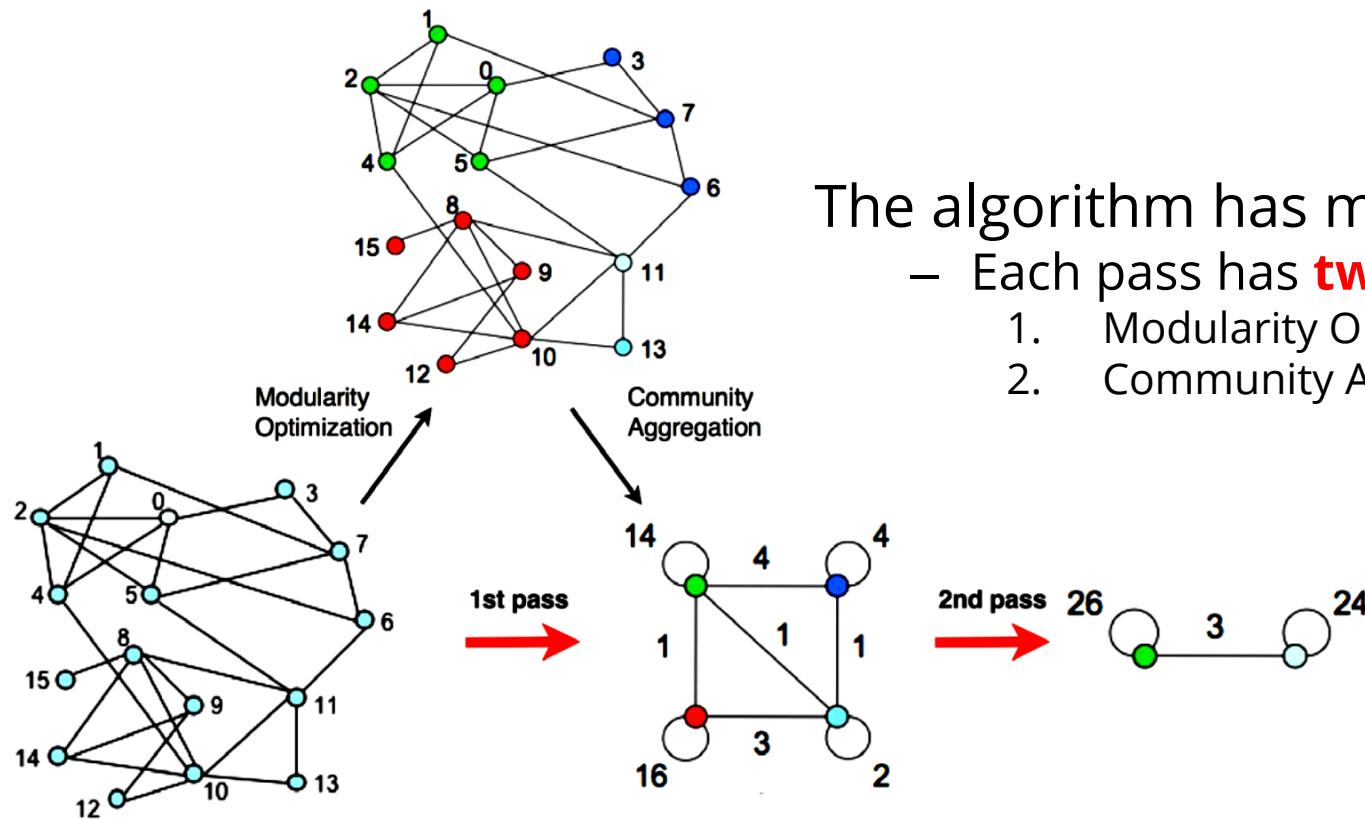
Modularity Matrix

Modularity Optimization for a Large Network

- Can be scaled up to billions of nodes
- Louvain Method
 - A greedy modularity optimization method for community detection
 - Invented by authors affiliated with *Université catholique de Louvain (UCL)*



Louvain Method



The algorithm has multiple passes

– Each pass has **two phases**

1. Modularity Optimization
2. Community Aggregation

Image from

Blondel, Vincent D., Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. "Fast unfolding of communities in large networks." *Journal of statistical mechanics: theory and experiment* 2008, no. 10 (2008): P10008.

Louvain Method

Start with a weighted network where all nodes are in their own communities (i.e., n communities)

First Phase:

- For each node v_i ,
 - For all neighbors $v_j \in N(v_i)$:
 - compute the modularity gain if v_i is removed from its community and placed in the community of v_j .
 - Find the community with the maximum modularity gain
 - If the maximum gain is positive, remove v_i from its community, and place it in that community
 - If no positive gain, do not change communities
- Repeat until no node changes its community

Important Points about Phase I

- A point can be considered multiple times
- A Local Minima of modularity maximization is achieved in phase I
- Phase I is order dependent
 - The modularity achieved is more or less stable and is less dependent on the initial order
 - The computation time depends on the initial order.



Second Phase:

- Build a new network
 - Nodes are communities
 - Edges are the edges between nodes in the corresponding communities (weights are sum of the weights)
 - Self-loops represent edges within the community
- The algorithm creates hierarchies of communities
- It usually ends in less than 10 passes
- It is seems to be an $O(n \log n)$ algorithm

Hierarchical Communities

- Earlier methods consider communities at a single level
 - Communities may have hierarchies.
 - Each community can have sub/super communities.
 - Hierarchical clustering deals with this scenario and generates community hierarchies.
- Initially n members are considered as either 1 or n communities in hierarchical clustering. These communities are gradually
 - merged (**agglomerative hierarchical clustering**) or
 - split (**divisive hierarchical clustering**)

Hierarchical Community Detection

- **Goal:** build a hierarchical structure of communities based on network topology
- Allow the analysis of a network at different resolutions
- Representative approaches:
 - Divisive Hierarchical Clustering
 - Agglomerative Hierarchical clustering

Divisive Hierarchical Clustering

- Divisive clustering
 - Partition nodes into several sets
 - Each set is further divided into smaller ones
 - Network-centric partition can be applied for the partition
- One particular example:

Girvan-Newman Algorithm: recursively remove the “weakest” links within a “community”

- Find the edge with the weakest link
 - Remove the edge and update the corresponding strength of each edge
- Recursively apply the above two steps until a network is discomposed into a desired number of connected components.
- Each component forms a community

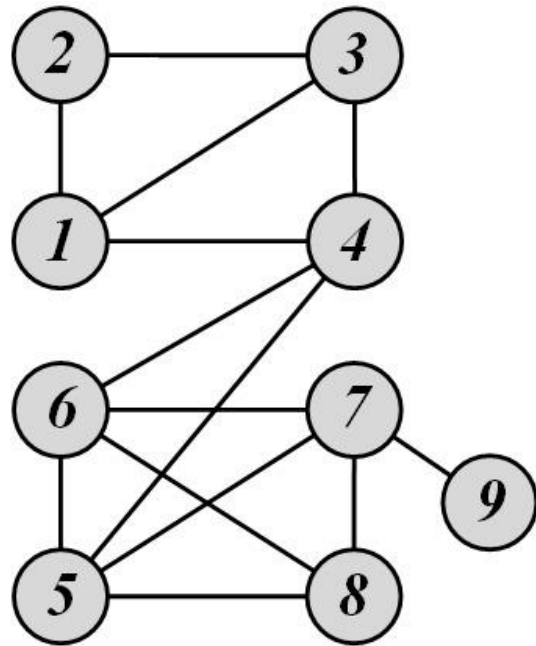
Edge Betweenness

- To determine weakest links, the algorithm uses **edge betweenness**.

Edge betweenness is the number of shortest paths that pass along with the edge

- Edge betweenness measures the “bridgeness” of an edge between two communities
- The edge with high betweenness tends to be the bridge between two communities.

Edge Betweenness: Example

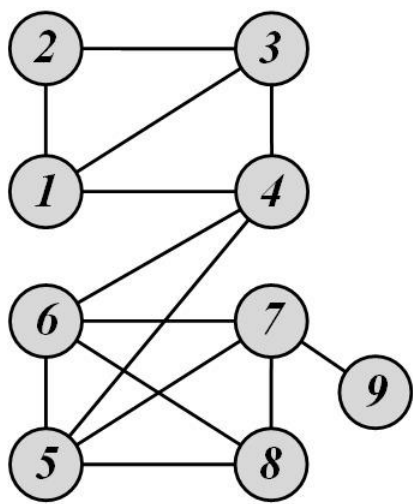


The edge betweenness of $e(1,2)$ is $6/2 + 1 = 4$, as all the shortest paths from 2 to $\{4, 5, 6, 7, 8, 9\}$ have to either pass $e(1,2)$ or $e(2,3)$, and $e(1,2)$ is the shortest path between 1 and 2

The Girvan-Newman Algorithm

1. Calculate edge betweenness for all edges in the graph.
2. Remove the edge with the highest betweenness.
3. Recalculate betweenness for all edges affected by the edge removal.
4. Repeat until all edges are removed.

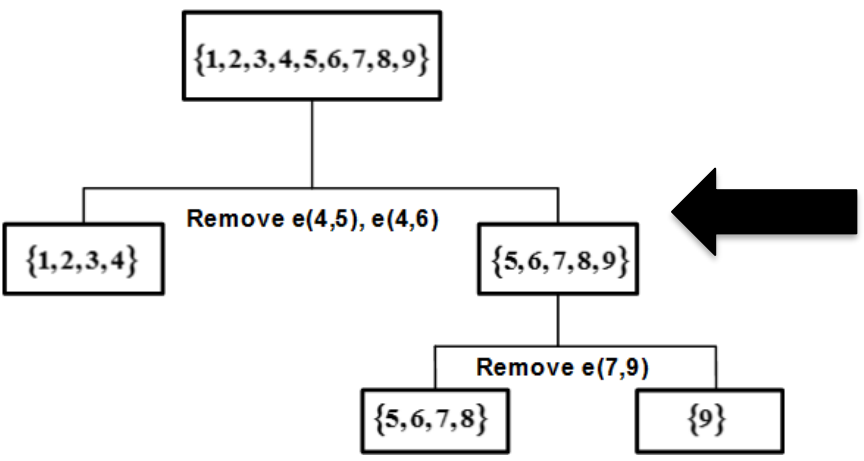
Edge Betweenness Divisive Clustering: Example



Initial betweenness value

	1	2	3	4	5	6	7	8	9
1	0	4	1	9	0	0	0	0	0
2	4	0	4	0	0	0	0	0	0
3	1	4	0	9	0	0	0	0	0
4	9	0	9	0	10	10	0	0	0
5	0	0	0	10	0	1	6	3	0
6	0	0	0	10	1	0	6	3	0
7	0	0	0	0	6	6	0	2	8
8	0	0	0	0	3	3	2	0	0
9	0	0	0	0	0	0	8	0	0

the first edge that needs to be removed is $e(4, 5)$ (or $e(4, 6)$)



By removing $e(4, 5)$, we compute the edge betweenness once again; this time, $e(4, 6)$ has the highest betweenness value: 20.

This is because all shortest paths between nodes $\{1,2,3,4\}$ to nodes $\{5,6,7,8,9\}$ must pass $e(4, 6)$; therefore, it has betweenness $4 \times 5 = 20$.